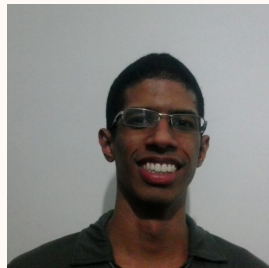


From V-JEPA to LeWorldModel

Accessible video world models:
understanding, prediction, and planning in latent space



Isabelle Rodrigues
PUC-Rio



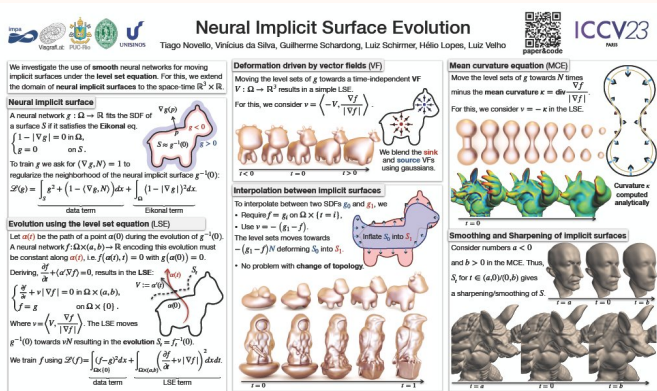
Vinicius da Silva
Tecgraf Institute

This tutorial grows from joint work with **Matheus Bessa, Felipe Kestelman, Alberto Sardinha and Alberto Raposo**. Thank you to the full survey team.

Our group works on **neural implicit representations** of geometry and images and, more recently, on latent world models. A short timeline:



Takeaway. The next three slides show three of these works in poster form; the last item on the timeline is what brings us to today's topic.



Novello, da Silva, Schardong, Schirmer, Lopes, Velho

Poster presented at ICCV 2023, Paris

- ▶ a smooth network g encodes a surface as its zero level set; a second network $f(p, t)$ encodes the **evolution** in space-time;
- ▶ the **level set equation** is the training loss, so the deformation is *learned*, not simulated;
- ▶ vector-field deformations, interpolation between shapes (topology changes included), mean-curvature smoothing and sharpening.

dsilvavinicius.github.io/nise

Schardong, Novello, Paz, Medvedev, da Silva, Velho, Gonçalves
Poster presented at CVPR 2024, Seattle

Neural Implicit Morphing of Face Images
Guilherme Schardong, Tiago Novello, Hallison Paz, Iuri Medvedev, Vinicius da Silva, Luiz Velho, Nuno Gonçalves

We investigate the use of smooth neural networks for morphing face images regularized by the **thin-plate energy**. For this, we model the time as a parameter and **disentangle** deformation from blending.

Neural Morphing Primer
A morphing between faces consists of a **warping** T of their domains for feature alignment and **blending** of the resulting warped faces.

We parametrize the **warping** by a network $T: \mathbb{R}^2 \times \mathbb{R} \rightarrow \mathbb{R}^2$ and train it using $\mathcal{L} = \lambda_1 \mathcal{W} + \lambda_2 \mathcal{B} + \lambda_3 \mathcal{P}$.

Warping constraint
 $\mathcal{W} = \int_0^1 \|T_x - M_x^T\|^2 dx + \int_0^1 \|T_y - M_y^T\|^2 dy$

Identity constraint
 $\mathcal{I} = \int_0^1 \|T_x - M_x^T\|^2 dx + \int_0^1 \|T_y - M_y^T\|^2 dy$

Data constraint
 $\mathcal{D} = \int_0^1 \|T_x - M_x^T\|^2 dx + \int_0^1 \|T_y - M_y^T\|^2 dy$

Thin-plate constraint
 $\mathcal{P} = \int_0^1 \|T_x - M_x^T\|^2 dx + \int_0^1 \|T_y - M_y^T\|^2 dy$

Finally, we blend the warped images $\mathcal{F}_i(\cdot, t)$ to define the morphing $\mathcal{F}$.

Poisson and generative blendings
We propose a **Poisson blending** where we align $\text{Jac}(\mathcal{F})$ with a vector field U , defined in terms of $\text{Jac}(\mathcal{F}_i)$. We optimize $\mathcal{B} = \int_0^1 \|\text{Jac}(\mathcal{F}) - U\|^2 dx + \int_0^1 \|U - \mathcal{F}^T\|^2 dx$.

Let $\mathcal{E}$ and $\mathcal{D}$ be encoder and decoder models. We embed the warped images $\mathcal{F}_i$ in the latent space: $\mathcal{E}(\mathcal{F}_i) = \mathcal{E}(\mathcal{F}_i, t)$. Then, we interpolate them using $\mathcal{F}_i(\cdot, t) = \mathcal{D}((1-t)\mathcal{E}_1(t) + t\mathcal{E}_2(t))$.

Poisson blending for feature transfer
Faces with different gender and ethnicity

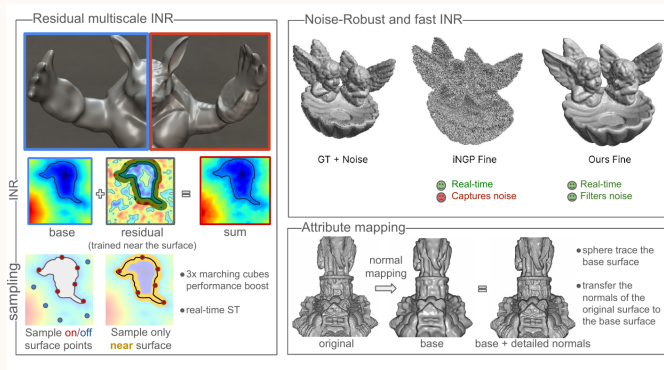
Generative methods demand aligned faces. Minor misalignments lead to mismatching.

Blending impacts morphing detection. Our warping+drAE blending is comparable to pure drAE, followed by StyleGAN2 and ours+5.Mix. Lastly classical warping and ours+0.Clone/linear

Non-linear warping leads to smoother alignment. Especially noticeable on videos

- ▶ a morphing is a **warping** $T: \mathbb{R}^2 \times \mathbb{R} \rightarrow \mathbb{R}^2$ of the image domains followed by a **blending** of the warped faces; time is a network input, so deformation and blending are disentangled;
- ▶ the warping is trained with landmark correspondences, identity/inverse constraints and a **thin-plate energy**;
- ▶ blending can be linear, Poisson (aligning the image Jacobian) or generative (diffusion autoencoder);
- ▶ the announcement went viral on X (237K views) and was quoted by Yann LeCun.

schardong.github.io/ifmorph



Neural Implicit Surfaces via Nested Multiscale Residuals

Accepted work at NeurIPS 2026

- ▶ the SDF is a **residual sum of MLPs** on nested near-surface bands: a coarse base plus finer residuals trained only where they matter;
- ▶ **noise-robust by construction**: the base keeps the shape where hash-grid methods reproduce the scan noise;
- ▶ real-time multiscale sphere tracing, a $3\times$ marching-cubes speed-up from near-surface sampling, and normal mapping of fine detail onto the base surface;
- ▶ best Chamfer distance and IoU on Stanford and Thingi32 with an order of magnitude fewer parameters.

1. build the intuition behind joint-embedding prediction;
2. open the model and inspect masks, tubelets, and collapse;
3. scale the idea to V-JEPA 2 and real video features;
4. add actions and plan through imagined latent futures;
5. revisit collapse through LeJEPA, SIGReg, and LeWorldModel;
6. end with research questions that fit academic-scale compute.

Our schedule

75 min intuition, mechanics + Lab 1 (15 min) → **30 min** coffee → **75 min** V-JEPA 2, planning, SIGReg + guided Lab 2 (5 min) + Lab 3 (10 min).

Imagine the following situations:

A ball leaves a hand.
Where will it be next?

A hand reaches for a cup.
What action will succeed?

Humans constantly make these predictions without rendering every possible image in our heads!
Intelligence is partly the ability to **imagine consequences before acting**.

If anticipation is part of intelligence, then an intelligent system should have some internal mechanism for asking:

Prediction

“What will probably happen if the world continues this way?”

Action-conditioned prediction

“What would happen if I acted differently?”

The second question is especially important for **planning and control**.

This is the motivation behind **world models**.

Why an internal model of the world?

803

Divide the role of an internal world model into three main capabilities.

Understanding

What is here? What is moving? What persists behind an occluder?

Anticipating

What is likely to happen next, even when the future is not fully visible?

Planning

How would the future change if the agent acted differently?

Notice that these form a **progression**! Understanding doesn't necessarily require intervention. Prediction adds temporal structure. Planning introduces agents decisions.

Takeaway. A world model is useful when its predictions inform decision-making, not merely when it produces plausible visual outcomes.

Video is a teacher that rarely gives labels

804

Why video?

Everyday video quietly reveals complex situations:

- ▶ objects that persist through motion and occlusion;
- ▶ how bodies, tools, and surfaces interact;
- ▶ which events tend to precede other events;
- ▶ how viewpoints change while the scene remains the same.

Large amount of information through time - requires lots of labeling and supervision if we choose traditional models.

The opportunity for context representation

By learning from videos, we can acquire representations containing semantic and physical structure before we ever introduce expensive action labels

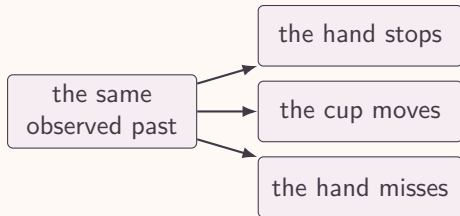
The future is not a single image

SIB

Let's get back to cup situation. "A hand reaches for a cup".

There is not necessarily one **correct future image**!

Given one short clip, several pixel-level futures can be equally plausible given the same context:



Problem: forcing a deterministic predictor to reproduce one exact future at the pixel level punishes it for not predicting details that were fundamentally uncertain (focus describing appearance).

Takeaway. Pixel prediction is unnecessarily sensitive to unpredictable detail. Latent prediction can abstract away some of that uncertainty while preserving structure useful downstream.

What should a prediction preserve?

306

What information should this representation preserve?

Usually important

- ▶ objects and their relations;
- ▶ pose, motion, and interaction-relevant state;
- ▶ what is possible next;
- ▶ what an action can change.

Sometimes incidental

- ▶ exact sensor noise;
- ▶ tiny texture fluctuations;
- ▶ codec artifacts;
- ▶ one arbitrary pixel realization.

The central problem is not simply compression. **It's learning the right level of abstraction.**

If the representation removes too little, prediction remains unnecessarily difficult. If it removes too much, it becomes useless downstream!

One scene, two learning problems

507

For the same scene, we might have two fundamentally **different learning objectives**:

Recreate what is missing

Ask a model to reconstruct what is missing, like the exact pixels or visual tokens. The model is rewarded for visual fidelity.

Predict a learned representation

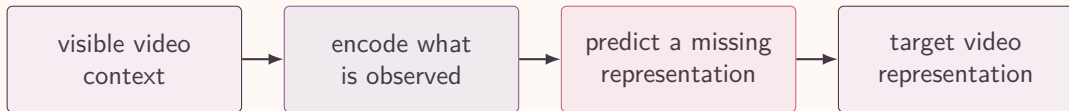
Recover the target representation of the missing content. The model can abstract away details that the learned representation does not preserve.

Takeaway. The same modern attention-based building blocks can support either problem; the learning target changes what knowledge is rewarded.

We will see that a JEPA is not required to reproduce every visual detail!

JEPA = Joint-Embedding Predictive Architecture

A JEPA learns to predict **representations** of missing or future content, rather than reconstructing the raw input itself.



For images, that target can correspond to masked image regions. For videos, targets become spatiotemporal.

Throughout this tutorial, we work with videos: the context and target are spatiotemporal regions, so the representation must capture motion and persistence — not only appearance.

There are several reasons to predict in representation space:

- ▶ many different pixel futures can share task-relevant structure;
- ▶ latent targets avoid spending capacity on every unpredictable visual detail;
- ▶ the encoder can focus capacity on motion, objects, and relations;
- ▶ the learned state can transfer to recognition, anticipation, and control;
- ▶ when used as a world model, compact latent states can make repeated rollouts much cheaper than pixel generation.

The trade-off

The trade-off is important: a JEPA does not naturally give us a visible imagined future, and if the representation throws away information that later matters, the predictor cannot recover it.

Summarizing **progression**:

Images

Predict the representation of a hidden region from visible context.

Video

Predict hidden spatiotemporal content and learn motion and persistence.

Control

Condition the latent prediction on an action and compare possible futures.

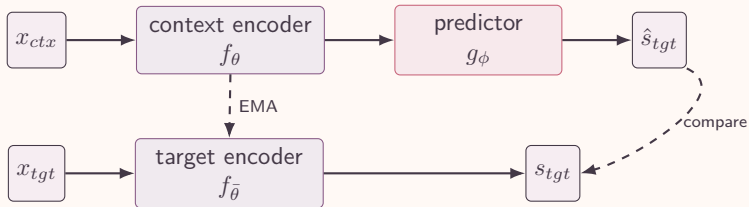
That is the transition from self-supervised representation learning toward an action conditioned world model.

...And this is the path from **I-JEPA** to **V-JEPA** to **JEPA world models**!

If the model does *not* need to draw the future,
what must its internal state still know?

Keep your answer. We will return to this through several different tests: recognition, physical-state probing, surprise, and planning.

Let's identify the EMA teacher–student recipe used by I-JEPA and V-JEPA.
Other JEPA variants can use different anti-collapse mechanisms.



- ▶ x_{ctx} is the **context view**: the information visible to the model.
- ▶ x_{tgt} is the **target view**; the information the model needs somehow to recover.
- ▶ f_{θ} is the **context encoder**, with trainable parameters θ .
- ▶ $f_{\bar{\theta}}$ is the **target encoder**; $\bar{\theta}$ follows θ by EMA, without gradient updates.
- ▶ g_{ϕ} is the **predictor network**, with trainable parameters ϕ .

The two branches have very different roles.

Top path: make a prediction

$$x_{ctx} \longrightarrow f_{\theta}(x_{ctx}) \longrightarrow g_{\phi}(\cdot, z) \longrightarrow \hat{s}_{tgt}$$

The model only sees the **context**.

It builds a representation of what it observed and asks:

*“Given what I could observe,
what should the target representation be?”*

Bottom path: provide the target

$$x_{tgt} \longrightarrow f_{\bar{\theta}}(x_{tgt}) \longrightarrow s_{tgt}$$

This branch has access to the **target view**.

It computes the representation that answers:

*“What representation
should I have predicted?”*

But this raises an important question:

Why not use f_{θ} on both sides?

Two encoders, two different roles

516

The context and target encoders serve different purposes during training.

Context encoder f_{θ}

Its job is to learn **useful representations** that allow the predictor to infer the target.

Its parameters θ are updated directly by **gradient descent / backpropagation**.

Target encoder $f_{\bar{\theta}}$

Its job is to provide a **stable target representation**.

It has its own current parameter values, $\bar{\theta}$.
They are **not independently optimized** by a second optimizer.

So how are the target encoder parameters updated?

They follow the context encoder slowly.

The target encoder is a moving teacher

5/7

Instead of gradient descent, the target encoder is updated by an **exponential moving average (EMA)**:

$$\bar{\theta} \leftarrow \tau \bar{\theta} + (1 - \tau) \theta$$

For example, if $\tau = 0.99$:

$$\bar{\theta}_{\text{new}} = 0.99 \bar{\theta}_{\text{old}} + 0.01 \theta$$

- ▶ θ may change directly after each gradient update.
- ▶ $\bar{\theta}$ moves only slightly toward θ , making the target representation **more stable**.

$f_{\bar{\theta}}$ is therefore a **slow-moving teacher**.

One more rule: stop-gradient on the target branch

8/8

There is one more important rule before we define the training objective.

During the backward pass, the two branches are treated differently.

$$\underbrace{x_{ctx} \rightarrow f_{\theta} \rightarrow g_{\phi} \rightarrow \hat{s}_{tgt}}_{\text{gradient can flow}}$$

$$\underbrace{x_{tgt} \rightarrow f_{\bar{\theta}} \rightarrow s_{tgt}}_{\text{gradient is stopped}}$$

- ▶ The prediction branch can be updated by backpropagation: this is how θ and ϕ learn.
- ▶ The target branch is **detached**: backpropagation does not update $\bar{\theta}$.
- ▶ $\bar{\theta}$ changes only through the **EMA update** from the previous slide.

Backpropagation: θ, ϕ **EMA:** $\bar{\theta}$

A schematic prediction objective for this EMA-based recipe is:

$$\mathcal{L}_{pred} = \left\| \underbrace{g_{\phi}(f_{\theta}(x_{ctx}), z)}_{\hat{s}_{tgt} : \text{prediction}} - \underbrace{\text{sg}(f_{\bar{\theta}}(x_{tgt}))}_{s_{tgt} : \text{target}} \right\|_2^2$$

- ▶ $g_{\phi}(f_{\theta}(x_{ctx}), z)$ is the **predicted target representation** $\hat{s}_{tgt}$.
- ▶ $f_{\bar{\theta}}(x_{tgt})$ is the **target representation** s_{tgt} .
- ▶ This squared distance illustrates the objective; V-JEPA 2 uses an L1 loss on masked target features.
- ▶ $\text{sg}(\cdot)$ means **stop-gradient**: the target branch is not updated by backpropagation.

JEPA predicts **representations of the target**, not the target pixels themselves.

One JEPA principle, different recipes

320

The architecture we just saw is an important JEPA recipe, but it is **not the global definition of all JEPA's variants**.

The central idea is to predict a **target representation** from a **context representation**.

$$\text{context} \longrightarrow \hat{s}_{tgt} \approx s_{tgt} \longleftarrow \text{target}$$

Different JEPA-based models can change:

- ▶ **what** representation is predicted;
- ▶ **what conditions** the prediction;
- ▶ and **how collapse / target stability** is handled.

Let's see how this evolves across models.

The first change is **what kind of context and target we predict**.

I-JEPA

Start from an **image**.

Some image regions form the context; other regions are targets.

$$x_{ctx} \longrightarrow \hat{s}_{tgt}$$

Predict the **representations of target regions**, not their pixels.

V-JEPA / V-JEPA 2

Now the input is a **video**.

Context and targets extend across space and time.

$$x_{ctx}^{video} \longrightarrow \hat{s}_{tgt}^{video}$$

Predict **masked video representations** in latent space.

In both cases, positional information tells the predictor **which target region should be predicted**.

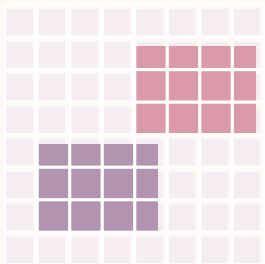
Same principle: **predict missing information in representation space**.

How does V-JEPA learn useful visual representations?

2023

Masked pretraining asks the model to predict **target representations from visible context**.

But **what we choose to hide** matters.



large contiguous targets

- ▶ Small isolated patches can often be inferred from nearby visual information.
- ▶ Large target blocks require **broader object and scene context**.
- ▶ In V-JEPA, masking extends across **space and time** in the video.

Masking shapes **what the representation must learn**.

This gives us useful latent representations of the observed world.

A glimpse of action-conditioned prediction

8/23

Masked video pretraining gives us a useful representation of **what is happening in the observed world**.

$$\text{video observation} \longrightarrow z_t$$

But for an intelligent agent, this is not enough.

“What will happen if I take this action?”

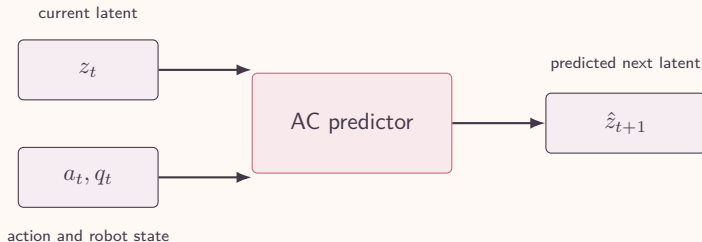
$$z_t, a_t \longrightarrow \hat{z}_{t+1}$$

Adding the action turns representation prediction into **latent dynamics prediction**.

V-JEPA 2-AC: predicting consequences of actions

824

V-JEPA 2-AC uses the pretrained V-JEPA 2 representation to model **how the world evolves under actions**.



- ▶ z_t comes from the visual encoder pretrained with **V-JEPA 2**.
- ▶ During action-conditioned post-training, this encoder is **frozen**.
- ▶ Here z_t denotes a **feature map**, not one compact vector. The actual predictor also receives the robot end-effector state.
- ▶ This diagram abbreviates the autoregressive prediction of future visual features.

We will get back to V-JEPA 2-AC later with some more details.

Why keep the predictor lightweight?

8/33

During pretraining, the encoder and predictor have **different jobs**.

Encoder f_θ

Learns the **reusable representation** of the input and should retain information that remains useful beyond the pretraining task.

$$x_{ctx} \longrightarrow f_\theta(x_{ctx})$$

Predictor g_ϕ

Learns the **conditional mapping** needed to recover the target representation. Keeping it smaller puts more pressure on the encoder to carry reusable information.

$$f_\theta(x_{ctx}) \longrightarrow \hat{s}_{tgt}$$

Most reusable capacity should live in the **encoder**, not in the pretraining predictor.

For downstream perception, the predictor can be discarded; the pretrained encoder is the reusable component.

A low prediction error is not enough

SS4

A prediction objective can be minimized by a **completely useless representation**.

Suppose the encoder maps every input to the same vector:

$$f_{\theta}(x) = c \quad \forall x$$

Then both the target and the prediction can become

$$s_{tgt} = c \quad \text{and} \quad \hat{s}_{tgt} = c.$$

So the prediction error is

$$\mathcal{L}_{pred} = \|\hat{s}_{tgt} - s_{tgt}\|_2^2 = \|c - c\|_2^2 = 0$$

Different inputs $\longrightarrow$ same representation.

Perfect prediction error, but the representation contains **no information about the input**.

This is representation collapse

8/26

If many different inputs are mapped to the same representation,

$$x_1, x_2, x_3, \dots \xrightarrow{f_\theta} c,$$

then the representation has **collapsed**.

What collapses?

Different inputs become almost indistinguishable in latent space.

Across samples:

$$\text{Var}(z) \rightarrow 0.$$

The representation loses **diversity and information**.

Why is this dangerous?

The prediction objective may still look excellent.

But the representation is useless for downstream tasks:

- ▶ recognition,
- ▶ classification,
- ▶ planning.

Low prediction loss $\neq$ useful representation learning.

A decreasing prediction loss is useful, but it only tells us whether the model is solving the **training objective**. We might want to ask extra questions:

- ▶ **Prediction loss by mask type:** are some target regions much easier or harder than others?
- ▶ **Gradient norms:** are gradients stable, or are they vanishing / exploding?
- ▶ **EMA momentum:** is the target encoder following the context encoder at the intended rate?
- ▶ **Throughput and memory:** is training computationally stable and efficient?

These signals tell us whether **optimization is behaving as expected**.

But they still do not answer the most important question:

Are we learning useful representations?

Monitoring pretraining: is the representation healthy?

5/28

Because low prediction error can coexist with collapse, we must also monitor the **representation itself**.

- ▶ **Feature standard deviation:** do features still vary across different inputs? Near-zero variance is a **collapse warning**.
- ▶ **Covariance / effective rank:** is information spread across many feature dimensions, or concentrated into only a few?
- ▶ **Nearest neighbors:** do nearby points in latent space correspond to meaningfully similar inputs?
- ▶ **Frozen probe:** freeze the encoder and train a small downstream head; can the representation already support a useful task?

Training objective health $\neq$ representation health.

A video batch is represented as a 5D tensor:

$$x \in \mathbb{R}^{B \times C \times T \times H \times W}$$

Batch and channels

- ▶ B : number of video clips in the batch;
- ▶ C : channels per frame ($C = 3$ for RGB).

Time and space

- ▶ T : number of frames;
- ▶ H, W : frame height and width.

V-JEPA does not pass individual pixels directly to the Transformer.
It first groups local regions across **space and time** into **tubelets**.

A tubelet covers a small region across **multiple consecutive frames**.

A 3D convolution uses kernel and stride

$$(\tau, P, P)$$

where

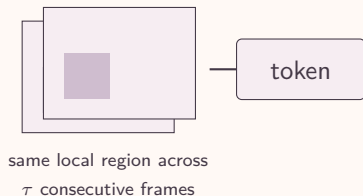
τ = frames per tubelet

and

$P \times P$ = spatial patch size.

Each tubelet becomes **one token** of dimension D .

A tubelet is the video analogue of an image patch.



After tokenization, V-JEPA chooses which tubelet tokens are **visible context** and which become **prediction targets**.

Smaller target blocks

Several spatial blocks (**8** in the original setup),
each covering roughly

15%

of the spatial token grid.

Larger target blocks

A few much larger blocks (**2** in the original
setup),
each covering roughly

70%

of the spatial token grid.

Crucially, each sampled spatial block is repeated across the **entire temporal dimension** of the clip.

So V-JEPA masks **spatial tubes through time**, not isolated patches from individual frames.
Their union produces a very high masking ratio (roughly 90% on average), limiting shortcuts from redundant video content.

So what should a good embedding support?

SSA

We have trained V-JEPA to predict masked representations.
But as we saw earlier, **low prediction error is not enough**.

Recognition

Freeze the encoder and use a small probe.

Can the embedding expose
objects and actions?

Structure

Can constrained readouts
recover

**scene, motion, pose,
or state information?**

Dynamics

As we move toward world
models:

can representations support
anticipation and planning?

A useful representation should make relevant information **easy to extract**.

Takeaway. The goal is not a visually attractive reconstructed frame; it is a useful representation of the world.

01_tiny_vjepa_from_scratch.ipynb

- ▶ generate moving-shape videos offline;
- ▶ tokenize with a 3D convolution;
- ▶ visualize a spatiotemporal block mask;
- ▶ train context encoder + predictor + EMA target;
- ▶ inspect loss, feature spread, and effective rank.

15 min

Google Colab

Coffee break

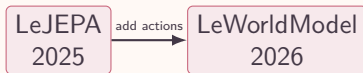
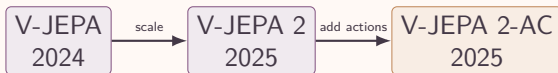
30 minutes · keep Colab connected

From video representations to world models

8:30

Before coffee, we learned how JEPA predicts **representations rather than pixels**, how V-JEPA builds those representations from video, and how its training recipe keeps them informative.

Now we follow two connected directions:



First: the frontier route

Scale representation learning, then learn **action-conditioned dynamics** on top of it.

Then: the lean route

Keep latent prediction, but ask whether the **training recipe itself can be simplified**.

V-JEPA 2 scales the representation before adding actions

336

V-JEPA 2 keeps the same broad idea that we studied before coffee, but scales it dramatically.

Large-scale pretraining

- ▶ more than **1 million hours** of internet video;
- ▶ large Vision Transformer encoders;
- ▶ no action labels required;
- ▶ representation learning remains self-supervised.

Resulting representation

- ▶ strong motion understanding;
- ▶ action recognition;
- ▶ action anticipation;
- ▶ transferable video features.

At this stage, V-JEPA 2 has learned
how to represent what is happening,
but not yet
what will happen if the agent acts.

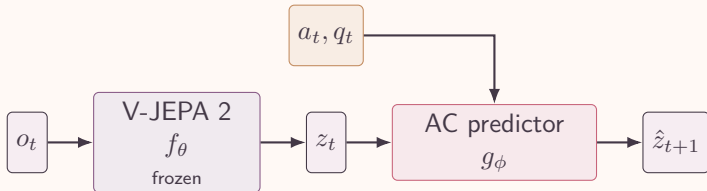
Back to AC modeling: dynamics on top of frozen perception

For V-JEPA 2-AC, the action-conditioned stage reuses the pretrained V-JEPA 2 representation.

Stage 1: perception

V-JEPA 2

- ▶ large-scale video SSL;
- ▶ learn visual state;
- ▶ then **freeze the encoder**.



Stage 2: dynamics

V-JEPA 2-AC

- ▶ robot interaction video;
- ▶ actions become inputs;
- ▶ train action- and end-effector-state-conditioned

$$z_t = f_\theta(o_t)$$

$$\hat{z}_{t+1} \approx g_\phi(z_{\leq t}, a_{\leq t}, q_{\leq t})$$

“Given past visual feature maps, actions and end-effector states q , what features come next?”

Latent MPC: imagine, score, act, repeat

838

Once the model can predict

$$(s_t, a_t) \longrightarrow \hat{s}_{t+1},$$

candidate actions can be rolled forward **entirely in latent space**:

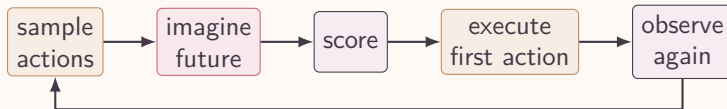
$$s_t \xrightarrow{a_t} \hat{s}_{t+1} \xrightarrow{a_{t+1}} \dots \xrightarrow{a_{t+H-1}} \hat{s}_{t+H}.$$

For V-JEPA 2-AC, the goal image is encoded into a feature map,

$$z_{\text{goal}} = f_{\theta}(o_{\text{goal}}),$$

and a candidate sequence is scored by

$$J = \|\hat{z}_{t+H} - z_{\text{goal}}\|_1$$



Frontier capability comes with a heavy recipe

839

V-JEPA 2 and V-JEPA 2-AC establish an important result:

latent prediction can support perception, anticipation, and planning.

But achieving that result involves several layers of machinery.

Scale

- ▶ very large video corpus;
- ▶ large encoder;
- ▶ expensive pretraining;
- ▶ separate robot post-training.

Training recipe

- ▶ context encoder;
- ▶ EMA target encoder;
- ▶ stop-gradient;
- ▶ predictor;
- ▶ schedules and multi-stage training.

Takeaway. The question is no longer whether latent prediction works.

The next question is whether we can obtain useful representations and dynamics with a **simpler training recipe**.

02_vjepa2_huggingface.ipynb

1. Load a released V-JEPA 2 checkpoint and one short video.
2. Run inference and inspect the **hidden-state tensor shapes**.
3. Compare representations for **forward vs. reverse temporal order**.

No training here.

The goal is to connect the architecture to an actual released V-JEPA 2 encoder.

10 min

Google Colab

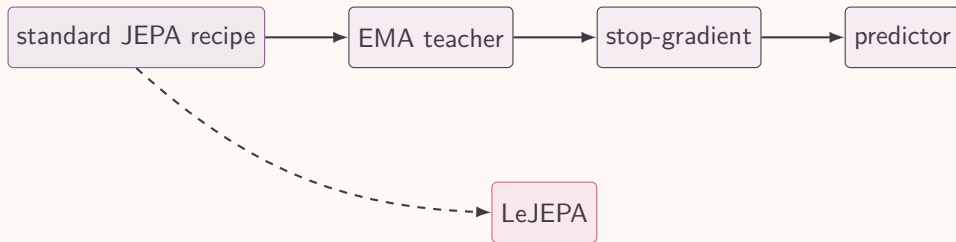
Now take the second branch: can we simplify the recipe?

543

We have just interacted with a real V-JEPA 2 representation.

Its quality is not the question.

The question is how much machinery was required to learn that representation.



Before coffee, we already established *why* representation collapse is dangerous.

LeJEPA asks a narrower and more useful question:

Can we keep the representation healthy without the asymmetric teacher–student recipe?

LeJEPA turns representation health into part of the objective

342

We discussed diagnostics such as:

feature variance, covariance, effective rank.

Those diagnostics ask: “*Is information still spread through the representation?*”

LeJEPA makes a related idea **part of training itself**.

Its theory identifies the isotropic Gaussian as the target embedding distribution:

$$z = f_{\theta}(x) \sim \mathcal{N}(0, I)$$

That target requires the embedding distribution to be

- ▶ centered;
- ▶ non-degenerate;
- ▶ spread across its dimensions.

Before coffee, distributional statistics were *diagnostics*. LeJEPA turns distributional structure into *regularization*.

Why an isotropic gaussian? (part I)

LeJEPA targets a standard isotropic Gaussian:

$$z \sim \mathcal{N}(0, I)$$

The first property is

$$\mathbb{E}[z] = 0$$

Thus,

$$\mathbb{E}[z] = \begin{bmatrix} \mathbb{E}[z_1] \\ \mathbb{E}[z_2] \\ \vdots \\ \mathbb{E}[z_d] \end{bmatrix} = 0.$$

It means that, across the data distribution, each latent dimension is centered around zero. The embedding cloud is **centered around the origin** and $\mathbb{E}[z] = 0$ controls the location of the embedding distribution.

Why an isotropic gaussian? (part II)

The second property of the target is

$$\text{Cov}(z) = I$$

This gives us two important conditions:

Unit variance

$$\text{Var}(z_i) = 1$$

Every latent dimension varies across samples.
A dimension cannot simply become constant.

Zero cross-covariance

$$\text{Cov}(z_i, z_j) = 0 \quad i \neq j$$

Different dimensions are decorrelated under the target distribution.

Together, these produce an **isotropic** covariance structure: no direction has larger variance than another.

Takeaway. $\text{Cov}(z) = I$ encourages variation to be distributed across the embedding dimensions instead of collapsing into a point, line, or low-rank subspace.

Why an isotropic gaussian? (part III)

So far we have required

$$\mathbb{E}[z] = 0 \quad \text{and} \quad \text{Cov}(z) = I.$$

But these two conditions do **not** uniquely determine the full distribution.

Many non-Gaussian distributions can also satisfy

$$\mathbb{E}[z] = 0, \quad \text{Cov}(z) = I.$$

Moment constraints

Mean and covariance describe only the first two moments: center + second-order spread.
They do not completely specify the shape of the distribution.

Gaussian target

LeJEPA instead targets the stronger condition

$$z \sim \mathcal{N}(0, I)$$

which specifies the **full embedding distribution**.

The LeJEPA theory connects this Gaussian target to its downstream prediction-risk analysis.

SIGReg turns the Gaussian target into a training objective

546

We now know the representation we would like to encourage:

$$z \sim \mathcal{N}(0, I)$$

But this raises a practical question: **How do we encourage a neural network to produce embeddings with this distribution?**

LeJEPA introduces a regularization module called SIGReg (Sketched Isotropic Gaussian Regularization).

Sketched

Instead of analyzing the full d -dimensional distribution, look at simpler random views of it.

Isotropic Gaussian

Those views should behave as expected if

$$z \sim \mathcal{N}(0, I).$$

Regularization

Turn the mismatch into a differentiable penalty added to the training loss.

SIGReg is the mechanism that translates “make the embedding distribution isotropic Gaussian” into something that gradient descent can optimize.

Suppose a batch gives us embeddings

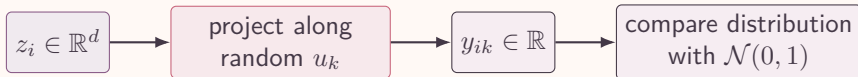
$$z_i \in \mathbb{R}^d.$$

Testing their full d -dimensional distribution directly would be expensive.

SIGReg instead samples random unit directions u_k and projects the embeddings onto them:

$$y_{ik} = u_k^\top z_i$$

Each direction turns a high-dimensional embedding into a **single scalar**.



Why a standard normal target? For $z \sim \mathcal{N}(0, I)$ and any unit vector u_k ,

$$u_k^\top z \sim \mathcal{N}(0, 1).$$

Takeaway. Instead of asking whether the entire embedding cloud is Gaussian, SIGReg repeatedly asks a much simpler question: *does this random 1-D view look Gaussian?*

From a random projection to a penalty

For each direction u_k , SIGReg needs a quantity that answers:

How far is this projected distribution from a standard Gaussian?

Let T_k denote the normality penalty computed for projection k .

$$T_k \text{ small} \implies \{y_{ik}\}_i \text{ looks approximately Gaussian}$$

$$T_k \text{ large} \implies \{y_{ik}\}_i \text{ deviates from } \mathcal{N}(0, 1)$$

Takeaway. Each random projection produces one scalar “how non-Gaussian is this view?” penalty.

SIGReg averages the penalties across random directions

SIGReg does not inspect only one direction.

It samples many random directions

$$u_1, u_2, \dots, u_M$$

and obtains one normality penalty for each.

The SIGReg loss aggregates these tests:

$$\mathcal{L}_{\text{SIGReg}}(Z) = \frac{1}{M} \sum_{k=1}^M T_k$$

During gradient descent,

$$\min_{\theta} \mathcal{L}_{\text{SIGReg}}$$

encourages the encoder to change its embeddings so that more and more random 1-D views resemble $\mathcal{N}(0, 1)$.

What changes from V-JEPA to LeJEPA?

345

The central prediction idea remains.

What changes is the mechanism used to keep the representation well behaved.

V-JEPA recipe

- ▶ context encoder;
- ▶ EMA target encoder;
- ▶ stop-gradient;
- ▶ predictor;
- ▶ latent prediction loss.

LeJEPA recipe

- ▶ shared trainable representation;
- ▶ predictive / invariance objective;
- ▶ SIGReg;
- ▶ no EMA teacher;
- ▶ no stop-gradient.

Schematically,

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda \mathcal{L}_{\text{SIGReg}}$$

LeJEPA does not change the problem we studied before coffee; it proposes a leaner solution to it.

The next question is now obvious: can dynamics be learned the same way?

V-JEPA 2-AC gave us strong frozen representation + action-conditioned predictor.

LeJEPA gave us representation learning without EMA / stop-gradient.

Putting those two ideas together suggests:

Can we learn perception and action-conditioned dynamics together, end-to-end?

That is the role of

LeWorldModel.

Takeaway. LeWorldModel is not a new definition of world modeling. It is the LeJEPA training principle applied to the action-conditioned dynamics problem we already saw with V-JEPA 2-AC.

LeWorldModel learns from transitions between observations

5/7

A training example contains three pieces:

Current observation
What the agent sees
before acting:
 o_t

Action
The intervention applied
to the environment:
 a_t

Next observation
What the agent sees *after*
that action:
 o_{t+1}

The learning problem is therefore:

Given **what I see now** and **what I do**,
predict a representation of **what I should see next**.

$$(o_t, a_t) \longrightarrow \text{representation of } o_{t+1}$$

LeWorldModel learns dynamics from ordinary visual transitions: before action, action, after action.

The observations are mapped into the **same learned latent space**:

$$\boxed{z_t = f_{\theta}(o_t)} \quad \boxed{z_{t+1} = f_{\theta}(o_{t+1})}$$

In the reported model, each frame contributes one projected [CLS] embedding,

$$\boxed{z_t \in \mathbb{R}^{192}}$$

latent vector.

LeWorldModel predictor and how it acts (part I)

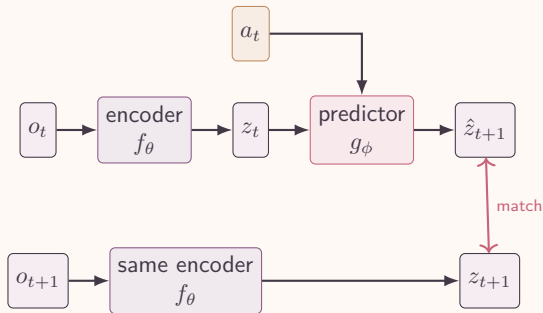
Schematically, the predictor receives latent history *and actions*:

$$\hat{z}_{t+1} \approx g_{\phi}(z_{\leq t}, a_{\leq t})$$

and tries to match the latent representation of what actually happened:

$$\hat{z}_{t+1} \approx z_{t+1}.$$

LeWorldModel predictor and how it acts (part II)



Takeaway. The model never has to reconstruct o_{t+1} : it predicts where the next observation should lie in the learned latent space.

Two losses train the complete LeWorldModel

343

The dynamics predictor should agree with the representation of the actual next observation:

$$\mathcal{L}_{\text{pred}} = \|\hat{z}_{t+1} - z_{t+1}\|_2^2$$

At the same time, SIGReg shapes the encoder's embedding distribution:

$$\mathcal{L}_{\text{LeWM}} = \mathcal{L}_{\text{pred}} + \lambda \mathcal{L}_{\text{SIGReg}}(Z)$$

Prediction

Encourages the state to preserve information needed to model how actions change the world.

SIGReg

Keeps the representation distribution non-degenerate while both sides learn together.

Takeaway. The same objective jointly determines **what the state represents** and **how that state evolves**.

V-JEPA 2-AC and LeWM differ mainly in how the state is obtained

5/19

Both ultimately learn

$$(z_t, a_t) \longrightarrow \hat{z}_{t+1}.$$

But they reach that equation through very different pipelines.

V-JEPA 2-AC

internet-scale video



V-JEPA 2



freeze representation



learn dynamics

LeWorldModel

task interaction video



pixels + actions



**learn representation
and dynamics jointly**

One asks the dynamics to adapt to a pretrained state space. The other allows the state space itself to adapt to the dynamics.

The same latent-MPC principle applies

SSB

We already learned the MPC/CEM planning loop with V-JEPA 2-AC. LeWorldModel uses the same basic idea, with its own latent cost.

Encode the current observation and visual goal:

$$z_t = f_\theta(o_t), \quad z_g = f_\theta(o_{\text{goal}}).$$

Roll candidate actions through the dynamics:

$$z_t \xrightarrow{a_t} \hat{z}_{t+1} \xrightarrow{a_{t+1}} \dots \xrightarrow{a_{t+H-1}} \hat{z}_{t+H}.$$

And search for actions minimizing

$$J = \|\hat{z}_{t+H} - z_g\|_2^2$$

using CEM.

Same MPC/CEM family; LeWM uses a squared-L2 terminal cost.

Different learned state space.

The LeWM contribution is primarily in *learning the world-model state*, not inventing a new planner.

One 192-D token makes repeated imagination cheap

66/87

Planning evaluates many candidate sequences over many future steps.
So representation size matters enormously.

Foundation-feature state

many spatial tokens

$$\{z_t^1, \dots, z_t^N\}$$

Each candidate future propagates a large feature set.

LeWorldModel state

one compact token

$$z_t \in \mathbb{R}^{192}$$

Each candidate future propagates one vector.

The authors report approximately $\sim 200\times$ fewer tokens per observation than DINO-WM.

Takeaway. Compression is especially valuable inside planning, because the state is evaluated repeatedly for thousands of hypothetical transitions.

The efficiency difference appears directly in planning time

353

The reported LeWorldModel is intentionally compact:

Model	Latent	Training
~ 15M parameters	192 dimensions / frame	1 GPU few hours

In the reported comparison,

$$\text{LeWM} \approx 1 \text{ s} \quad \text{vs.} \quad \text{DINO-WM} \approx 47 \text{ s}$$

for a planning cycle, corresponding to up to approximately $48\times$ faster planning.

The simpler representation changes the cost of the complete *train + imagine + plan* loop.

Does the compact model still solve control tasks?

853

LeWM is evaluated on four environments:

Two-Room

2D navigation

Reacher

2-joint arm

Push-T

object pushing

OGBench-Cube

3D manipulation

The result is deliberately mixed:

- ▶ LeWM surpasses DINO-WM on **Push-T and Reacher**;
- ▶ DINO-WM retains an advantage on **OGBench-Cube**;
- ▶ LeWM is weaker on **Two-Room**.

Takeaway. The result is not “small beats large everywhere.”

A compact end-to-end model can remain competitive while making planning dramatically cheaper.

Can we recover physical state from the learned latent?

SS4

The world model is trained from **pixels and actions**, not from simulator state.

After training, freeze the encoder and train a small supervised probe:

$$z_t \longrightarrow \text{probe} \longrightarrow \text{physical variable}$$

On Push-T, the probes try to recover:

Agent position	Block position	Block angle
(x_a, y_a)	(x_b, y_b)	θ_b
Where is the pusher?	Where is the T-shaped object?	How is the object oriented?

The physical variables supervise only the **probe**. They were not used to train the LeWorldModel representation.

If a simple probe can recover physical state from z_t , then that information survived inside the learned representation.

The same prediction error becomes a surprise signal

8/8

A world model also defines an expectation about what should happen next.

The predicted next state is

$$\hat{z}_{t+1} = g_{\phi}(z_t, a_t),$$

while the real observation gives

$$z_{t+1} = f_{\theta}(o_{t+1}).$$

So we can define

$$r_t = \|\hat{z}_{t+1} - z_{t+1}\|_2$$

as a simple surprise score.

Prediction error becomes a violation-of-expectation signal

The comparison is not between two arbitrary observations.

It is conditioned on both the current state and the action that was actually taken:

$$(z_t, a_t) \longrightarrow \hat{z}_{t+1} \quad \text{vs.} \quad o_{t+1} \longrightarrow z_{t+1}.$$

So the surprise score

$$\mathcal{S}_t = \|\hat{z}_{t+1} - z_{t+1}\|_2$$

asks: **“Given what I saw and what I did, did the world evolve as I expected?”**

Expected transition

The observed consequence is close to the model's prediction:

$$z_{t+1} \approx \hat{z}_{t+1}$$

$\Rightarrow \mathcal{S}_t$ is small.

Unexpected transition

The observed consequence differs from the predicted dynamics:

$$z_{t+1} \not\approx \hat{z}_{t+1}$$

$\Rightarrow \mathcal{S}_t$ increases.

Reading the transitions

Important interpretation

A large score means *unexpected according to the learned world model*.

It does not automatically mean “physically impossible.”

Takeaway. Prediction error can therefore act as a **violation-of-expectation signal**: the model detects when reality departs from the dynamics it has learned.

LeWorldModel also reveals where the simple recipe breaks

356

LeWM does not dominate every environment.
That is scientifically useful.

Scale

Experiments remain far smaller
than internet-scale video worlds.

Capacity

One 192-D token may be
insufficient for complex
multi-object worlds.

Geometry

The poor Two-Room result
suggests that one global latent
geometry may not suit every
control problem.

The last point is particularly interesting:

the question is no longer
“did the representation collapse?”

but

“what non-collapsed geometry is best for the task?”

03_latent_world_model _and_planning.ipynb

Now we reproduce the essential LeWM loop on a small system:

- ▶ build (o_t, a_t, o_{t+1}) transitions;
- ▶ train compact latent dynamics;
- ▶ compare prediction-only with sliced-Gaussian regularization;
- ▶ probe whether position survives in the latent;
- ▶ use the learned dynamics inside latent MPC.

10 min

Offline-friendly

After LeWorldModel, the same simplification moves back to video pretraining

333

LeWorldModel showed that the LeJEPA recipe can support **compact action-conditioned dynamics**.

But there is another direction: **Can the same simpler training principle scale to general video representation learning?**

That is the goal of **LeVJEPA**.

Important distinction

LeVJEPA is a self-supervised **video encoder**.

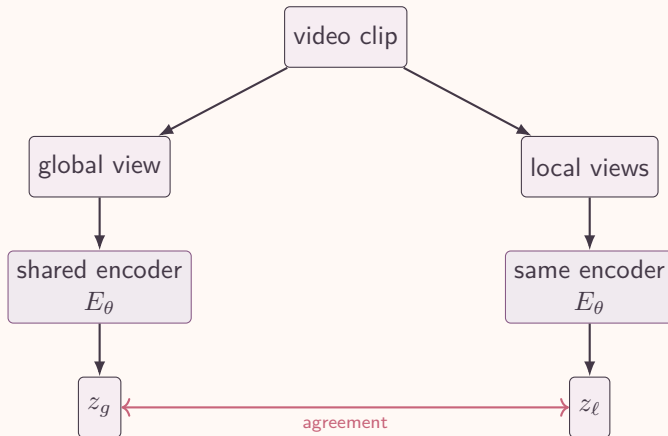
It is not itself an action-conditioned world model or an MPC controller.

Takeaway. LeWorldModel and LeVJEPA inherit the same LeJEPA/SIGReg idea, but apply it to different problems.

LeVJEPA needs only one shared encoder

859

Instead of context and EMA-target branches, LeVJEPA constructs spatially and photometrically different views of the same temporal clip.



LeVJEPA objective (part I)

LeVJEPA combines view agreement and distributional regularization:

$$\mathcal{L} = \mathcal{L}_{\text{inv}} + \lambda \mathcal{L}_{\text{SIGReg}}.$$

- ▶ $\mathcal{L}_{\text{inv}}$: the global and local views of the same temporal clip should have similar projected [CLS] embeddings.
- ▶ $\mathcal{L}_{\text{SIGReg}}$: the embeddings are encouraged toward an isotropic Gaussian distribution, excluding constant solutions.

Gradients flow through all views of the shared encoder and projector.

LeVJEPA objective (part II)

The coefficient λ controls the trade-off between **view agreement** and **distributional regularization**.

No target encoder. No EMA. No stop-gradient. No predictor network. (A shared projector is trained and discarded after pretraining.)

Takeaway. $\mathcal{L}_{\text{inv}}$ tells different views *what should stay the same*; SIGReg constrains *what the embedding space should look like*. Together, they provide the complete LeVJEPA training signal.

Without reconstruction, most video tokens can disappear

380

LeVJEPa never asks: *“Can you reconstruct the missing patch?”*

So patch tokens do not all need to enter the Transformer.

The main recipe randomly discards 95% of patch tokens before the encoder.

This plays two roles:

Compute

Far fewer tokens enter self-attention, dramatically reducing each forward pass.

Augmentation

Different sparse subsets of the same clip must lead to compatible representations.

In the reported ablation, ImageNet-1K attentive probing rises from

33.9% → 47.6%

between no dropping and 95% dropping.

Once reconstruction disappears, “seeing less” can simultaneously **reduce compute and strengthen representation invariance**.

The simpler video recipe changes the compute trade-off

302

At matched training epochs on the same data, LeVJEPA reports comparable or better performance than V-JEPA 2 across several model scales while using $5.6\times$ to $20.8\times$ less total pretraining compute.

For a ViT-B comparison with identical data and total pretraining FLOPs (frozen evaluation; IN1K/SSv2 attentive probes, K400 linear probe):

Method	IN1K	SSv2	K400
VideoMAEv2	53.4	43.6	37.4
V-JEPA 2	51.6	42.5	40.7
LeVJEPA	61.0	40.4	44.6

LeVJEPA does not dominate every benchmark: different evaluations emphasize different aspects of representation quality.

Takeaway. The main result is not universal benchmark dominance; it is a major shift in the **compute–representation trade-off**.

LeWorldModel and LeVJEPA use the same principle for different jobs

300

	LeWorldModel	LeVJEPA
Goal	action-conditioned dynamics	general video representation learning
Training input	(o_t, a_t, o_{t+1})	multiple views of video clips
Learning relation	$(z_t, a_t) \rightarrow z_{t+1}$	global/local representation agreement
SIGReg	Yes	Yes
Actions	Yes	No
Latent MPC	Yes	No
Large-scale video SSL	Not the primary goal	Yes

Takeaway. LeJEPA is the shared training principle.
LeWorldModel and LeVJEPA are two different instantiations of that principle.

The theoretical question has also moved beyond collapse

2024

Once representations remain non-degenerate, a stronger question becomes possible:

Does the learned representation actually recover the hidden state of the world?

A 2026 theoretical analysis studies worlds with stationary additive-noise dynamics. For Gaussian latents and the specified stationary additive-noise dynamics, at the theoretical optimum LeJEPa recovers the latents **up to an orthogonal transformation**:

$$z_t = Q s_t, \quad Q^\top Q = I$$

where s_t is the world state; Q permits rotations and reflections.

Important scope

This is an identifiability guarantee for a specified class of dynamical systems. It does not imply that arbitrary real video automatically yields the true physical state.

Takeaway. The frontier is shifting from “how do we avoid collapse?” to “when does the representation recover the structure of the world?”

For planning, non-collapse is still not the whole story

83

The first half of the tutorial gave us a binary failure mode:

collapsed vs. non-collapsed.

But control needs more than simply “not collapsed.”

The geometry must also preserve distinctions that matter to the task:

- ▶ which states are physically different?
- ▶ which differences affect future dynamics?
- ▶ which distances are meaningful to the planner?
- ▶ how many independent latent directions does the task actually require?

This gives a more interesting question:

How much latent dimensionality and isotropy does a particular world model actually need?

Takeaway. Representation health is not only about preventing collapse; it is also about matching latent geometry to task structure.

Open challenges: the target itself is a research question

SS24

The **distributional target** raises questions beyond scale, capacity and transfer:

Right target for planning?

LeJEPa's optimality is about downstream *probes*; planning needs latents whose geometry supports prediction and search. LeWM's own Two-Room failure hints that the two can conflict.

A test is not a guarantee

LeWM samples 1,024 directions per step. Finite minibatches and directions do not certify that the full joint distribution is Gaussian. Matching mean and covariance alone is weaker still.

How much isotropy is enough?

Prediction pulls one way and SIGReg the other; training settles at an equilibrium effective rank that nobody chooses, monitors or reports.

What should we measure?

Effective rank, sliced Gaussianity, certificates: which diagnostics actually predict planning success, and which are blind to it?

Takeaway. All of these are testable at academic scale: LeWM is public, has 15M parameters and trains in hours on one GPU.

Our current investigation: just-enough isotropy in LeWorldModel

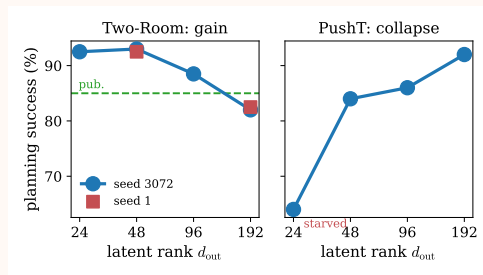
8/22

Question. What happens when the isotropic-Gaussian target is *actually met*, and how much isotropy does planning need?

Instrument. A **Gaussian-neuron** (RBF) head whose activation moments are closed-form: exact whitening by construction, per-direction Gaussianity diagnostics, and effective-rank diagnostics for any latent.

Findings so far (LeWM, its own 10-epoch budget):

- ▶ in our runs, SIGReg effective rank is 67–99 of 192;
- ▶ forcing compliance hurts: Two-Room planning falls 82 → 28%;
- ▶ dialing latent rank to the task repairs Two-Room (82 → 93%) and correctly flips on PushT (64 vs. 92%);
- ▶ effective rank orders planning success (Spearman -0.8); SIGReg's own statistic carries none (0.0).



Planning success vs. latent rank d_{out} at LeWM's 10-epoch budget (dashed: published LeWM). Manuscript in preparation (2026).

Our results suggest a task-dependent trade-off between prediction and isotropy. We make that trade-off **measurable and adjustable**.

1. Diagnostics for other world models

Audit V-JEPA 2, DINO-WM and EB-JEPA latents with effective rank, sliced Gaussianity and certificates. Does just-enough isotropy hold beyond LeWM?

2. Rank-aware regularizers

Replace a fixed Gaussian target by a rank-adaptive one (PCA + subspace SIGReg, learned dials) and design terms that see *joint* non-Gaussianity, which finite random 1-D projection sets may miss.

3. Graphics-native world models

Our renderers and neural implicit assets (evolving surfaces, morphs, Gaussian-splat scenes) as controllable simulators with known intrinsic dimension: natural testbeds for the rank story.

4. Theory that predicts practice

Closed-form moments and certificates for other activation families; planning signal-to-noise vs. effective rank; what “task demand” means for geometry-heavy scenes.

Takeaway. Every branch runs on public code at one-GPU scale. If one of them matches your interests, talk to us during the break or reach us at <https://dsilvavinicius.github.io>.



Thank you

Questions, ideas, or collaborations?